

CLEYA.AI - COMPLETE REPLIT PROMPT

Fix All Issues + Build New Features | 90-Day Implementation

🎯 OBJECTIVE

Transform Cleya.ai from current state (5.8/10) to production-ready platform (7.5+/10) by fixing all critical issues and implementing missing core features.

- Timeline:** 90 days
- Team:** 2-3 full-stack developers
- Budget:** 15-20 Lakhs
- Expected Outcome:** 25-50L MRR, 20K users, 0.4 viral coefficient

📁 PROJECT STRUCTURE

```
cleya-ai-platform/
├─ frontend/                # Next.js 14 + TypeScript + Tailwind
│  ├─ app/
│  │  ├─ (marketing)/       # Landing pages
│  │  ├─ (dashboard)/      # User dashboard
│  │  ├─ api/              # API routes
│  │  └─ layout.tsx
│  ├─ components/
│  │  ├─ ui/               # shadcn/ui components
│  │  ├─ forms/            # Form components
│  │  ├─ messaging/        # Chat components
│  │  ├─ matching/         # Match cards, explanations
│  │  └─ scheduling/       # Calendar components
│  ├─ lib/
│  │  ├─ utils.ts
│  │  ├─ api.ts            # API client
│  │  └─ constants.ts
│  └─ styles/
├─ backend/                 # Node.js + Express + TypeScript
│  ├─ src/
│  │  ├─ config/
│  │  ├─ controllers/
│  │  ├─ models/
│  │  ├─ services/
│  │  ├─ middleware/
│  │  ├─ utils/
│  │  └─ app.ts
│  └─ tests/
├─ shared/                  # Shared types, constants
│  └─ types/
├─ infrastructure/         # Terraform, Docker
└─ docs/
```

🔴 PHASE 1: CRITICAL FIXES (Days 1-14)

1.1 SEO & Metadata Fixes (Day 1-2)

PRIORITY: P0 - CRITICAL

```
// frontend/app/layout.tsx - FIX THESE IMMEDIATELY

export const metadata = {
  // ❌ WRONG - CURRENT
  // metadataBase: new URL('http://localhost:5000'),

  // ✅ CORRECT - FIX TO THIS
  metadataBase: new URL('https://cleya.ai'),

  title: 'Cleya.ai - AI Superconnector for Indian Startups',
  description: 'Meet the right investors, founders, and operators. AI-powered warm intros for India\'s
  ↳ startup ecosystem.',

  openGraph: {
    type: 'website',
    url: 'https://cleya.ai',
    siteName: 'Cleya.ai',
    // ❌ WRONG - CURRENT
    // images: [{ url: 'http://localhost:5000/og-image.svg' }],

    // ✅ CORRECT - FIX TO THIS
    images: [
      {
        url: 'https://cleya.ai/og-image.png',
        width: 1200,
        height: 630,
        alt: 'Cleya.ai - AI Superconnector for Indian Startups'
      }
    ],
    locale: 'en_IN',
  },

  twitter: {
    card: 'summary_large_image',
    site: '@cleyaai',
    creator: '@cleyaai',
    // ❌ WRONG - CURRENT
    // images: ['http://localhost:5000/og-image.svg'],

    // ✅ CORRECT
    images: ['https://cleya.ai/og-image.png'],
  },

  robots: {
```

```

index: true,
follow: true,
googleBot: {
  index: true,
  follow: true,
  'max-video-preview': -1,
  'max-image-preview': 'large',
  'max-snippet': -1,
},
},

// Add canonical URLs
alternates: {
  canonical: 'https://cleya.ai',
},

// Add verification tags
verification: {
  google: 'YOUR_GOOGLE_SEARCH_CONSOLE_VERIFICATION_CODE',
},
}

```

Tasks:

- Replace all localhost:5000 references with cleya.ai
- Replace all boardy-ai-platform.replit.app references with cleya.ai
- Generate proper OG image (1200x630 PNG)
- Add canonical tags to all pages
- Submit sitemap to Google Search Console
- Add robots.txt with correct sitemap URL

```

# public/robots.txt
User-agent: *
Allow: /

Sitemap: https://cleya.ai/sitemap.xml

# Disallow admin routes
Disallow: /admin/
Disallow: /api/internal/

```

1.2 Security Fixes (Day 3-5)

PRIORITY: P0 - CRITICAL

```

// backend/src/config/security.ts

import helmet from 'helmet';

```

```

import rateLimit from 'express-rate-limit';
import cors from 'cors';

// Remove X-Powered-By header
app.disable('x-powered-by');

// Helmet security headers
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"], // Remove unsafe-inline eventually
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"],
      connectSrc: ["'self'", "https://api.cleya.ai"],
      fontSrc: ["'self'"],
      objectSrc: ["'none'"],
      mediaSrc: ["'self'"],
      frameSrc: ["'none'"],
    },
  },
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
  referrerPolicy: {
    policy: 'strict-origin-when-cross-origin',
  },
})));

// Rate limiting
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: 'Too many requests from this IP, please try again later.',
  standardHeaders: true,
  legacyHeaders: false,
});

app.use('/api/', limiter);

// Stricter rate limiting for auth endpoints
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5, // 5 attempts per 15 minutes
  skipSuccessfulRequests: true,
});

app.use('/api/auth/', authLimiter);

```

```
// CORS configuration
app.use(cors({
  origin: process.env.NODE_ENV === 'production'
    ? ['https://cleya.ai', 'https://www.cleya.ai']
    : ['http://localhost:3000'],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

Environment Variables:

```
# Rotate these immediately
LINKEDIN_CLIENT_ID=NEW_CLIENT_ID_HERE
LINKEDIN_CLIENT_SECRET=NEW_CLIENT_SECRET_HERE
JWT_SECRET=GENERATE_NEW_STRONG_SECRET
ENCRYPTION_KEY=GENERATE_32_CHAR_KEY
```

Tasks:

- Rotate all OAuth credentials
- Remove dev domain from all configs
- Add rate limiting to all endpoints
- Implement CSP headers
- Add security headers (HSTS, X-Frame-Options, etc.)
- Remove 'unsafe-eval' from CSP
- Add admin authentication middleware

1.3 Performance Optimization (Day 6-8)

PRIORITY: P0 - CRITICAL

```
// frontend/next.config.js

const nextConfig = {
  // Enable compression
  compress: true,

  // Image optimization
  images: {
    formats: ['image/webp', 'image/avif'],
    minimumCacheTTL: 60,
    domains: ['cleya.ai', 'cdn.cleya.ai'],
  },

  // Enable static optimization
  output: 'standalone',
```

```

// Headers for caching
async headers() {
  return [
    {
      source: '/*:all*(svg|jpg|png|woff2)',
      locale: false,
      headers: [
        {
          key: 'Cache-Control',
          value: 'public, max-age=31536000, immutable',
        },
      ],
    },
    {
      source: '/*:path*',
      headers: [
        {
          key: 'Cache-Control',
          value: 'public, max-age=3600, stale-while-revalidate=86400',
        },
      ],
    },
  ];
},
};

module.exports = nextConfig;

```

```

// backend/src/config/compression.ts

import compression from 'compression';

app.use(compression({
  level: 6, // Balanced compression
  filter: (req, res) => {
    if (req.headers['x-no-compression']) {
      return false;
    }
    return compression.filter(req, res);
  },
}));

```

Tasks:

- Enable Brotli/Gzip compression
- Add HTML caching headers
- Optimize images (WebP format)
- Implement CDN (Cloudflare/CloudFront)
- Add resource hints (preconnect, preload)

- Reduce JS bundle size (tree shaking, code splitting)
- Lazy load below-fold content

1.4 Trust Signals (Day 9-11)

PRIORITY: P0 - CRITICAL

```
// frontend/app/about/page.tsx

export default function AboutPage() {
  return (
    <div className="max-w-4xl mx-auto py-16 px-4">
      <h1 className="text-4xl font-bold mb-8">About Cleya.ai</h1>

      {/* Company Info */}
      <section className="mb-12">
        <h2 className="text-2xl font-semibold mb-4">Our Mission</h2>
        <p className="text-gray-600 mb-4">
          Cleya.ai is building the AI-powered infrastructure for India's startup ecosystem.
          We're on a mission to help 100,000 founders raise capital, find co-founders, and
          hire talent by 2030.
        </p>
        <div className="grid grid-cols-3 gap-4 mt-8">
          <div className="text-center">
            <div className="text-3xl font-bold text-teal-600">7,500+</div>
            <div className="text-sm text-gray-500">Members</div>
          </div>
          <div className="text-center">
            <div className="text-3xl font-bold text-teal-600">50Cr+</div>
            <div className="text-sm text-gray-500">Funding Facilitated</div>
          </div>
          <div className="text-center">
            <div className="text-3xl font-bold text-teal-600">1,200+</div>
            <div className="text-sm text-gray-500">Successful Intros</div>
          </div>
        </div>
      </section>

      {/* Team Section */}
      <section className="mb-12">
        <h2 className="text-2xl font-semibold mb-4">Our Team</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          <TeamMember
            name="Rahul Sharma"
            role="CEO & Co-founder"
            bio="Ex-Sequoia, IIT Bombay. Built 2 startups."
            image="/team/rahul.jpg"
            linkedin="https://linkedin.com/in/rahulsharma"
          />
          <TeamMember
```

```

        name="Priya Patel"
        role="CTO & Co-founder"
        bio="Ex-Google AI, CMU. ML researcher."
        image="/team/priya.jpg"
        linkedin="https://linkedin.com/in/priyapatel"
    />
    <TeamMember
        name="Arun Kumar"
        role="Head of Product"
        bio="Ex-Flipkart, ISB. Product leader."
        image="/team/arun.jpg"
        linkedin="https://linkedin.com/in/arunkumar"
    />
</div>
</section>

{/* Investors/Backers */}
<section className="mb-12">
    <h2 className="text-2xl font-semibold mb-4">Backed By</h2>
    <div className="flex gap-8 items-center">
        
        
        
    </div>
</section>

{/* Contact Info */}
<section>
    <h2 className="text-2xl font-semibold mb-4">Contact Us</h2>
    <div className="space-y-2 text-gray-600">
        <p>📍 Bangalore, India</p>
        <p>✉️ hello@cleya.ai</p>
        <p>☎️ +91 80 1234 5678</p>
    </div>
</section>
</div>
);
}

```

```
// frontend/app/pricing/page.tsx - CREATE THIS PAGE
```

```

export default function PricingPage() {
    return (
        <div className="max-w-6xl mx-auto py-16 px-4">
            <div className="text-center mb-12">
                <h1 className="text-4xl font-bold mb-4">Simple, Transparent Pricing</h1>
                <p className="text-xl text-gray-600">
                    Start free. Upgrade when you're ready.
                </p>
            </div>

```



```

<div className="grid grid-cols-1 md:grid-cols-3 gap-8">
  {/* Free Tier */}
  <PricingCard
    name="Free"
    price="₹0"
    period="forever"
    description="For founders exploring the ecosystem"
    features=[
      '3 AI matches per month',
      '1 warm intro per month',
      '50 connections',
      'Basic profile',
      'Community access',
    ]
    cta="Get Started Free"
    highlighted={false}
  />

  {/* Professional */}
  <PricingCard
    name="Professional"
    price="₹999"
    period="per month"
    description="For active founders raising capital"
    features=[
      '15 AI matches per month',
      '5 warm intros per month',
      '200 connections',
      'Rich profile with pitch deck',
      'Priority matching',
      'Analytics dashboard',
      'Email support',
    ]
    cta="Start 14-Day Free Trial"
    highlighted={true}
  />

  {/* Growth */}
  <PricingCard
    name="Growth"
    price="₹2,999"
    period="per month"
    description="For funded startups scaling fast"
    features=[
      'Unlimited AI matches',
      '20 warm intros per month',
      'Unlimited connections',
      'Verified badge',
      'Top search ranking',
      'Advanced analytics',
      'Priority support',
    ]
  />

```

```

        'CRM integration',
    ]}
    cta="Contact Sales"
    highlighted={false}
  />
</div>

{/* Enterprise */}
<div className="mt-12 bg-gray-50 rounded-lg p-8 text-center">
  <h3 className="text-2xl font-semibold mb-2">Enterprise</h3>
  <p className="text-gray-600 mb-4">
    For VC firms and accelerators. Custom solutions.
  </p>
  <ul className="text-left max-w-md mx-auto mb-6 space-y-2">
    <li>Unlimited team members</li>
    <li>API access</li>
    <li>Custom matching algorithms</li>
    <li>Dedicated account manager</li>
    <li>SLA guarantee</li>
  </ul>
  <Button size="lg">Contact Sales</Button>
</div>

{/* FAQ */}
<div className="mt-16">
  <h2 className="text-2xl font-semibold mb-6 text-center">Frequently Asked Questions</h2>
  <FAQ />
</div>
</div>
);
}

```

Tasks:

- Create /about page with team photos, mission, stats
- Create /pricing page with 3 tiers + enterprise
- Add real team photos (not stock)
- Display real metrics (users, funding facilitated)
- Add investor/backer logos
- Add contact information (address, email, phone)
- Create FAQ page

1.5 User Verification (Day 12-14)

PRIORITY: P0 - CRITICAL

```
// backend/src/services/verification.ts
```

```

import { User } from '../models/User';
import { linkedInAPI } from '../linkedin';

export class VerificationService {

  // LinkedIn OAuth verification
  async verifyLinkedIn(userId: string, accessToken: string) {
    try {
      const profile = await linkedInAPI.getProfile(accessToken);

      await User.findByIdAndUpdate(userId, {
        'verification.linkedin': {
          verified: true,
          profileUrl: profile.vanityName
            ? `https://linkedin.com/in/${profile.vanityName}`
            : null,
          verifiedAt: new Date(),
        },
        'profile.headline': profile.headline,
        'profile.industry': profile.industry,
        'profile.location': profile.location?.name,
      });

      return { success: true };
    } catch (error) {
      return { success: false, error: 'LinkedIn verification failed' };
    }
  }

  // Email domain verification for corporate emails
  async verifyEmailDomain(userId: string, email: string) {
    const allowedDomains = [
      'sequoiacap.com',
      'blume.vc',
      'elevationcapital.com',
      'matrixpartners.in',
      'accel.com',
      'bessemervp.com',
      // Add more VC/startup domains
    ];

    const domain = email.split('@')[1];
    const isVerified = allowedDomains.includes(domain);

    await User.findByIdAndUpdate(userId, {
      'verification.emailDomain': {
        verified: isVerified,
        domain,
        verifiedAt: new Date(),
      },
    });
  }
}

```

```

    return { success: true, isVerified };
}

// SEBI accreditation verification for investors
async verifySEBI(userId: string, sebiNumber: string) {
    // Integrate with SEBI API or manual verification
    // This is a placeholder - implement actual SEBI check
    const isValid = await this.validateSEBINumber(sebiNumber);

    await User.findByIdAndUpdate(userId, {
        'verification.sebi': {
            verified: isValid,
            registrationNumber: sebiNumber,
            verifiedAt: new Date(),
        },
    });

    return { success: true, isValid };
}

// Calculate verification score
async getVerificationScore(userId: string): Promise<number> {
    const user = await User.findById(userId);
    let score = 0;

    if (user.verification?.linkedin?.verified) score += 30;
    if (user.verification?.emailDomain?.verified) score += 20;
    if (user.verification?.sebi?.verified) score += 40;
    if (user.verification?.phone?.verified) score += 10;

    return score;
}
}

```

```

// frontend/components/verification/VerificationBadge.tsx

interface VerificationBadgeProps {
    score: number;
    showDetails?: boolean;
}

export function VerificationBadge({ score, showDetails }: VerificationBadgeProps) {
    const getBadgeColor = () => {
        if (score >= 80) return 'bg-green-100 text-green-800 border-green-200';
        if (score >= 50) return 'bg-yellow-100 text-yellow-800 border-yellow-200';
        return 'bg-gray-100 text-gray-800 border-gray-200';
    };

    const getBadgeText = () => {
        if (score >= 80) return 'Highly Verified';
    };
}

```

```

    if (score >= 50) return 'Verified';
    return 'Basic';
  };

  return (
    <div className={`inline-flex items-center gap-2 px-3 py-1 rounded-full border ${getBadgeColor()}`}>
      <Shield className="w-4 h-4" />
      <span className="text-sm font-medium">{getBadgeText()}</span>
      {showDetails && <span className="text-xs opacity-75">({score}%)</span>}
    </div>
  );
}

```

Tasks:

- Implement LinkedIn OAuth verification
- Add email domain verification
- Create verification score algorithm
- Add verification badges to profiles
- Show verification details on hover
- Prioritize verified users in matching

🟡 PHASE 2: CORE FEATURES (Days 15-45)

2.1 In-App Messaging System (Days 15-25)

PRIORITY: P0 - CRITICAL (Platform stickiness)

```

// shared/types/messaging.ts

export interface Message {
  id: string;
  conversationId: string;
  senderId: string;
  recipientId: string;
  content: string;
  contentType: 'text' | 'image' | 'file' | 'meeting_invite';
  metadata?: {
    fileUrl?: string;
    fileName?: string;
    fileSize?: number;
    meetingId?: string;
  };
  status: 'sent' | 'delivered' | 'read';
  createdAt: Date;
  updatedAt: Date;
}

```

```

export interface Conversation {
  id: string;
  participants: string[]; // User IDs
  participantRoles: {
    userId: string;
    role: 'founder' | 'investor' | 'operator';
  }[];
  lastMessage?: Message;
  unreadCount: Record<string, number>; // userId -> count
  matchId?: string; // Reference to the match that started this
  status: 'active' | 'archived' | 'blocked';
  createdAt: Date;
  updatedAt: Date;
}

```

```

// backend/src/services/messaging.ts

import { Server } from 'socket.io';
import { Message, Conversation } from '../models';
import { notificationService } from '../notifications';

export class MessagingService {
  private io: Server;

  constructor(io: Server) {
    this.io = io;
    this.setupSocketHandlers();
  }

  private setupSocketHandlers() {
    this.io.on('connection', (socket) => {
      const userId = socket.handshake.auth.userId;

      // Join user's room for direct messages
      socket.join(`user:${userId}`);

      // Join all conversation rooms
      this.getUserConversations(userId).then((conversations) => {
        conversations.forEach((conv) => {
          socket.join(`conversation:${conv.id}`);
        });
      });

      // Handle new message
      socket.on('message:send', async (data) => {
        try {
          const message = await this.sendMessage({
            senderId: userId,
            ...data,
          });
        }
      });
    });
  }
}

```

```

    // Broadcast to conversation room
    this.io.to(`conversation:${data.conversationId}`).emit('message:received', message);

    // Send notification to recipient if offline
    const recipientId = data.recipientId;
    const recipientSocket = this.io.sockets.adapter.rooms.get(`user:${recipientId}`);

    if (!recipientSocket || recipientSocket.size === 0) {
      await notificationService.sendPushNotification(recipientId, {
        title: 'New Message',
        body: `You have a new message from ${message.senderName}`,
        data: { conversationId: data.conversationId },
      });
    }
  } catch (error) {
    socket.emit('message:error', { error: error.message });
  }
});

// Handle typing indicator
socket.on('typing:start', (data) => {
  socket.to(`conversation:${data.conversationId}`).emit('typing:start', {
    userId,
    conversationId: data.conversationId,
  });
});

socket.on('typing:stop', (data) => {
  socket.to(`conversation:${data.conversationId}`).emit('typing:stop', {
    userId,
    conversationId: data.conversationId,
  });
});

// Handle read receipts
socket.on('message:read', async (data) => {
  await this.markMessagesAsRead(data.conversationId, userId);
  socket.to(`conversation:${data.conversationId}`).emit('message:read', {
    userId,
    conversationId: data.conversationId,
    readAt: new Date(),
  });
});

socket.on('disconnect', () => {
  console.log(`User ${userId} disconnected`);
});
}

async sendMessage(data: {

```

```

senderId: string;
recipientId: string;
conversationId?: string;
content: string;
contentType?: string;
metadata?: any;
}): Promise<Message> {
  // Get or create conversation
  let conversationId = data.conversationId;

  if (!conversationId) {
    const conversation = await this.getOrCreateConversation(
      data.senderId,
      data.recipientId
    );
    conversationId = conversation.id;
  }

  // Create message
  const message = await Message.create({
    conversationId,
    senderId: data.senderId,
    recipientId: data.recipientId,
    content: data.content,
    contentType: data.contentType || 'text',
    metadata: data.metadata,
    status: 'sent',
  });

  // Update conversation last message
  await Conversation.findByIdAndUpdate(conversationId, {
    lastMessage: message,
    $inc: { [`unreadCount.${data.recipientId}`]: 1 },
    updatedAt: new Date(),
  });

  return message;
}

async getOrCreateConversation(userId1: string, userId2: string): Promise<Conversation> {
  // Check if conversation exists
  let conversation = await Conversation.findOne({
    participants: { $all: [userId1, userId2], $size: 2 },
  });

  if (!conversation) {
    conversation = await Conversation.create({
      participants: [userId1, userId2],
      unreadCount: { [userId1]: 0, [userId2]: 0 },
      status: 'active',
    });
  }
}

```



```

    }

    return conversation;
  }

  async markMessagesAsRead(conversationId: string, userId: string): Promise<void> {
    await Message.updateMany(
      {
        conversationId,
        recipientId: userId,
        status: { $ne: 'read' },
      },
      { status: 'read' }
    );

    await Conversation.findByIdAndUpdate(conversationId, {
      $set: { [`unreadCount.${userId}`]: 0 },
    });
  }

  async getUserConversations(userId: string): Promise<Conversation[]> {
    return Conversation.find({
      participants: userId,
      status: 'active',
    })
      .populate('lastMessage')
      .populate('participants', 'name profile.photo role')
      .sort({ updatedAt: -1 });
  }
}

```

```

// frontend/components/messaging/ChatInterface.tsx

'use client';

import { useEffect, useRef, useState } from 'react';
import { useSocket } from '@hooks/useSocket';
import { MessageBubble } from './MessageBubble';
import { MessageInput } from './MessageInput';
import { ConversationHeader } from './ConversationHeader';

interface ChatInterfaceProps {
  conversationId: string;
  recipient: {
    id: string;
    name: string;
    photo?: string;
    role: string;
    company?: string;
  };
};

```

```

export function ChatInterface({ conversationId, recipient }: ChatInterfaceProps) {
  const [messages, setMessages] = useState<Message[]>([]);
  const [isTyping, setIsTyping] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);
  const socket = useSocket();

  useEffect(() => {
    if (!socket) return;

    // Join conversation room
    socket.emit('conversation:join', { conversationId });

    // Load messages
    fetch(`/api/conversations/${conversationId}/messages`)
      .then((res) => res.json())
      .then((data) => setMessages(data.messages));

    // Listen for new messages
    socket.on('message:received', (message: Message) => {
      setMessages((prev) => [...prev, message]);
      scrollToBottom();
    });

    // Listen for typing indicators
    socket.on('typing:start', () => setIsTyping(true));
    socket.on('typing:stop', () => setIsTyping(false));

    // Listen for read receipts
    socket.on('message:read', (data) => {
      setMessages((prev) =>
        prev.map((msg) =>
          msg.senderId === data.userId ? { ...msg, status: 'read' } : msg
        )
      );
    });
  });

  return () => {
    socket.off('message:received');
    socket.off('typing:start');
    socket.off('typing:stop');
    socket.off('message:read');
  };
}, [socket, conversationId]);

const scrollToBottom = () => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
};

const handleSendMessage = (content: string, contentType = 'text') => {
  if (!socket) return;

```

```

socket.emit('message:send', {
  conversationId,
  recipientId: recipient.id,
  content,
  contentType,
});
});

return (
  <div className="flex flex-col h-full bg-white rounded-lg shadow-lg">
    <ConversationHeader recipient={recipient} />

    <div className="flex-1 overflow-y-auto p-4 space-y-4">
      {messages.map((message, index) => (
        <MessageBubble
          key={message.id}
          message={message}
          isOwn={message.senderId !== recipient.id}
          showAvatar={index === 0 || messages[index - 1].senderId !== message.senderId}
        />
      ))}

      {isTyping && (
        <div className="flex items-center gap-2 text-gray-500 text-sm">
          <div className="flex gap-1">
            <span className="w-2 h-2 bg-gray-400 rounded-full animate-bounce" />
            <span className="w-2 h-2 bg-gray-400 rounded-full animate-bounce delay-100" />
            <span className="w-2 h-2 bg-gray-400 rounded-full animate-bounce delay-200" />
          </div>
          <span>{recipient.name} is typing...</span>
        </div>
      )}

      <div ref={messagesEndRef} />
    </div>

    <MessageInput onSend={handleSendMessage} />
  </div>
);
}

```

Tasks:

- Set up Socket.io server
- Create Message and Conversation models
- Build chat interface components
- Implement real-time messaging
- Add typing indicators

- Add read receipts
 - Add file/image sharing
 - Add push notifications
 - Message persistence and history
-

2.2 Meeting Scheduling (Days 26-32)

PRIORITY: P1 - HIGH

```
// backend/src/services/scheduling.ts

import { google } from 'googleapis';
import { Calendar } from '../models';

export class SchedulingService {

  async createMeeting(data: {
    hostId: string;
    guestId: string;
    title: string;
    description?: string;
    duration: number; // minutes
    proposedSlots: Date[];
  }) {
    // Create meeting request
    const meeting = await Calendar.create({
      hostId: data.hostId,
      guestId: data.guestId,
      title: data.title,
      description: data.description,
      duration: data.duration,
      proposedSlots: data.proposedSlots,
      status: 'pending',
    });

    // Send notification to guest
    await notificationService.sendNotification(data.guestId, {
      type: 'meeting_request',
      title: 'New Meeting Request',
      body: `You have a meeting request from ${host.name}`,
      data: { meetingId: meeting.id },
    });

    return meeting;
  }

  async confirmMeeting(meetingId: string, selectedSlot: Date, userId: string) {
    const meeting = await Calendar.findById(meetingId);
```

```

    if (!meeting) throw new Error('Meeting not found');

    // Verify user is part of meeting
    if (meeting.hostId !== userId && meeting.guestId !== userId) {
        throw new Error('Unauthorized');
    }

    meeting.selectedSlot = selectedSlot;
    meeting.status = 'confirmed';
    await meeting.save();

    // Create calendar events for both parties
    await this.createCalendarEvent(meeting);

    // Send confirmation notifications
    const otherPartyId = meeting.hostId === userId ? meeting.guestId : meeting.hostId;
    await notificationService.sendNotification(otherPartyId, {
        type: 'meeting_confirmed',
        title: 'Meeting Confirmed',
        body: `Your meeting has been scheduled for ${selectedSlot.toLocaleString()}`,
        data: { meetingId: meeting.id },
    });

    return meeting;
}

private async createCalendarEvent(meeting: any) {
    // Create Google Calendar event (if integrated)
    // Or generate ICS file for email

    const eventDetails = {
        summary: meeting.title,
        description: meeting.description,
        start: {
            dateTime: meeting.selectedSlot.toISOString(),
            timeZone: 'Asia/Kolkata',
        },
        end: {
            dateTime: new Date(meeting.selectedSlot.getTime() + meeting.duration * 60000).toISOString(),
            timeZone: 'Asia/Kolkata',
        },
        attendees: [
            { email: meeting.hostEmail },
            { email: meeting.guestEmail },
        ],
        conferenceData: {
            createRequest: {
                requestId: meeting.id,
                conferenceSolutionKey: { type: 'hangoutsMeet' },
            },
        },
    },

```

```

    };

    // Send ICS file via email to both parties
    await emailService.sendMeetingInvite(eventDetails);
  }
}

```

```

// frontend/components/scheduling/MeetingScheduler.tsx

interface MeetingSchedulerProps {
  recipientId: string;
  recipientName: string;
  onScheduled?: () => void;
}

export function MeetingScheduler({ recipientId, recipientName, onScheduled }: MeetingSchedulerProps) {
  const [duration, setDuration] = useState(30);
  const [selectedDates, setSelectedDates] = useState<Date[]>([]);
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');

  const handleSubmit = async () => {
    const response = await fetch('/api/meetings', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        guestId: recipientId,
        title: title || `Meeting with ${recipientName}`,
        description,
        duration,
        proposedSlots: selectedDates,
      }),
    });

    if (response.ok) {
      toast.success('Meeting request sent!');
      onScheduled?.();
    } else {
      toast.error('Failed to send meeting request');
    }
  };

  return (
    <div className="p-6 bg-white rounded-lg shadow-lg max-w-md">
      <h3 className="text-lg font-semibold mb-4">Schedule a Meeting</h3>

      <div className="space-y-4">
        <div>
          <label className="block text-sm font-medium mb-1">Meeting Title</label>
          <Input
            value={title}

```

```

        onChange={(e) => setTitle(e.target.value)}
        placeholder={`Meeting with ${recipientName}`}
      />
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Duration</label>
      <Select value={duration.toString()} onChange={(v) => setDuration(Number(v))}>
        <SelectTrigger>
          <SelectValue />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="15">15 minutes</SelectItem>
          <SelectItem value="30">30 minutes</SelectItem>
          <SelectItem value="45">45 minutes</SelectItem>
          <SelectItem value="60">1 hour</SelectItem>
        </SelectContent>
      </Select>
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Propose Times</label>
      <DateTimePicker
        multiple
        value={selectedDates}
        onChange={setSelectedDates}
        minDate={new Date()}
        maxSelections={3}
      />
      <p className="text-xs text-gray-500 mt-1">
        Select up to 3 time slots. {recipientName} will choose one.
      </p>
    </div>

    <div>
      <label className="block text-sm font-medium mb-1">Description (Optional)</label>
      <Textarea
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        placeholder="What would you like to discuss?"
        rows={3}
      />
    </div>

    <Button onClick={handleSubmit} className="w-full" disabled={selectedDates.length === 0}>
      Send Meeting Request
    </Button>
  </div>
</div>
);

```

```
}
```

Tasks:

- Create Calendar model
- Build meeting scheduler component
- Implement time slot selection
- Add Google Calendar integration
- Generate ICS files for email
- Add Zoom/Google Meet link generation
- Send meeting reminders
- Handle timezone conversion (IST default)

2.3 Rich User Profiles (Days 33-38)

PRIORITY: P1 - HIGH

```
// shared/types/profile.ts

export interface UserProfile {
  // Basic Info
  userId: string;
  name: string;
  photo?: string;
  headline?: string;
  bio?: string;

  // Role-specific fields
  role: 'founder' | 'investor' | 'operator' | 'advisor';

  // For Founders
  company?: {
    name: string;
    stage: 'idea' | 'mvp' | 'pre-seed' | 'seed' | 'series-a' | 'series-b+';
    sector: string[];
    website?: string;
    pitchDeck?: string;
    traction?: {
      revenue?: string;
      users?: string;
      growth?: string;
    };
  };
  teamSize?: number;
  location: string;
  fundingRaised?: string;
};
```



```

// For Investors
investmentPreferences?: {
  stages: string[];
  sectors: string[];
  checkSize: {
    min: number;
    max: number;
  };
  geography: string[];
  thesis?: string;
  portfolio?: Array<{
    companyName: string;
    sector: string;
    stage: string;
  }>;
};

// For Operators
experience?: {
  currentRole?: string;
  company?: string;
  yearsOfExperience: number;
  skills: string[];
  achievements?: string[];
  lookingFor: 'full-time' | 'contract' | 'advisory' | 'co-founder';
  expectedCompensation?: {
    equity: boolean;
    salary?: string;
  };
};

// Social Links
linkedIn?: string;
twitter?: string;
website?: string;

// Verification
verificationScore: number;
isVerified: boolean;

// Activity
joinedAt: Date;
lastActive: Date;
matchCount: number;
introCount: number;
}

```

```

// frontend/components/profile/ProfileCard.tsx

```

```

interface ProfileCardProps {
  profile: UserProfile;
}

```

```

matchScore?: number;
showActions?: boolean;
onConnect?: () => void;
onMessage?: () => void;
onSchedule?: () => void;
}

export function ProfileCard({
  profile,
  matchScore,
  showActions = true,
  onConnect,
  onMessage,
  onSchedule,
}: ProfileCardProps) {
  return (
    <div className="bg-white rounded-lg shadow-md overflow-hidden hover:shadow-lg transition-shadow">
      <div className="relative h-24 bg-gradient-to-r from-teal-500 to-teal-600">
        <div className="absolute -bottom-10 left-4">
          <Avatar className="w-20 h-20 border-4 border-white">
            <AvatarImage src={profile.photo} />
            <AvatarFallback>{profile.name.charAt(0)}</AvatarFallback>
          </Avatar>
        </div>

        {matchScore && (
          <div className="absolute top-4 right-4 bg-white/90 backdrop-blur px-3 py-1 rounded-full">
            <span className="text-sm font-semibold text-teal-700">{matchScore}% Match</span>
          </div>
        )}
      </div>

      <div className="pt-12 px-4 pb-4">
        <div className="flex items-start justify-between">
          <div>
            <h3 className="text-lg font-semibold">{profile.name}</h3>
            <p className="text-gray-600 text-sm">{profile.headline}</p>
          </div>
          <VerificationBadge score={profile.verificationScore} />
        </div>

        <div>
          <div className="mt-3 space-y-1">
            <div className="flex items-center gap-2 text-sm">
              <Building className="w-4 h-4 text-gray-400" />
              <span className="font-medium">{profile.company.name}</span>
              <Badge variant="secondary">{profile.company.stage}</Badge>
            </div>

```

```

<div className="flex items-center gap-2 text-sm text-gray-600">
  <MapPin className="w-4 h-4" />
  <span>{profile.company.location}</span>
</div>
<div className="flex flex-wrap gap-1 mt-2">
  {profile.company.sector.map((s) => (
    <Badge key={s} variant="outline" className="text-xs">{s}</Badge>
  ))}
</div>
</div>
))

{profile.role === 'investor' && profile.investmentPreferences && (
  <div className="mt-3 space-y-1">
    <div className="flex items-center gap-2 text-sm">
      <Wallet className="w-4 h-4 text-gray-400" />
      <span>Check size: ₹{profile.investmentPreferences.checkSize.min}L -
        ₹{profile.investmentPreferences.checkSize.max}L</span>
    </div>
    <div className="flex flex-wrap gap-1 mt-2">
      {profile.investmentPreferences.sectors.map((s) => (
        <Badge key={s} variant="outline" className="text-xs">{s}</Badge>
      ))}
    </div>
  </div>
)}

{/* Bio */}
{profile.bio && (
  <p className="mt-3 text-sm text-gray-600 line-clamp-3">{profile.bio}</p>
)}

{/* Actions */}
{showActions && (
  <div className="flex gap-2 mt-4">
    <Button size="sm" onClick={onMessage} className="flex-1">
      <MessageCircle className="w-4 h-4 mr-2" />
      Message
    </Button>
    <Button size="sm" variant="outline" onClick={onSchedule}>
      <Calendar className="w-4 h-4" />
    </Button>
  </div>
)}
</div>
</div>
);
}

```

Tasks:

- Expand user profile schema
- Create profile editing interface
- Add pitch deck upload
- Add portfolio companies for investors
- Add skills/experience for operators
- Create profile completeness score
- Add profile preview

2.4 Match Explanation Cards (Days 39-42)

PRIORITY: P1 - HIGH

```
// frontend/components/matching/MatchExplanation.tsx

interface MatchExplanationProps {
  matchScore: number;
  reasons: Array<{
    factor: string;
    score: number;
    description: string;
    icon: React.ReactNode;
  }>;
}

export function MatchExplanation({ matchScore, reasons }: MatchExplanationProps) {
  const [isExpanded, setIsExpanded] = useState(false);

  return (
    <div className="bg-gradient-to-br from-teal-50 to-blue-50 rounded-lg p-4 border border-teal-100">
      <div className="flex items-center justify-between mb-3">
        <div className="flex items-center gap-2">
          <Sparkles className="w-5 h-5 text-teal-600" />
          <span className="font-semibold text-teal-900">Why this match?</span>
        </div>
        <button
          onClick={() => setIsExpanded(!isExpanded)}
          className="text-sm text-teal-600 hover:text-teal-700"
        >
          {isExpanded ? 'Hide' : 'Show details'}
        </button>
      </div>

      <div className="flex items-center gap-4 mb-4">
        <div className="relative w-16 h-16">
          <svg className="w-full h-full transform -rotate-90">
            <circle
              cx="32"

```

```

        cy="32"
        r="28"
        stroke="currentColor"
        strokeWidth="4"
        fill="transparent"
        className="text-teal-200"
      />
    <circle
      cx="32"
      cy="32"
      r="28"
      stroke="currentColor"
      strokeWidth="4"
      fill="transparent"
      strokeDasharray={2 * Math.PI * 28}
      strokeDashoffset={2 * Math.PI * 28 * (1 - matchScore / 100)}
      className="text-teal-600 transition-all duration-1000"
    />
  </svg>
  <div className="absolute inset-0 flex items-center justify-center">
    <span className="text-lg font-bold text-teal-700">{matchScore}%</span>
  </div>
</div>
<div>
  <p className="text-sm text-gray-600">
    Based on {reasons.length} factors including sector fit, stage alignment, and investment
    ↪ thesis.
  </p>
</div>
</div>

{isExpanded && (
  <div className="space-y-3 mt-4">
    {reasons.map((reason, index) => (
      <div key={index} className="flex items-start gap-3 bg-white/50 rounded-lg p-3">
        <div className="text-teal-600 mt-0.5">{reason.icon}</div>
        <div className="flex-1">
          <div className="flex items-center justify-between mb-1">
            <span className="font-medium text-sm">{reason.factor}</span>
            <span className="text-sm font-semibold text-teal-600">+{reason.score}%</span>
          </div>
          <p className="text-xs text-gray-600">{reason.description}</p>
        </div>
      </div>
    ))}
  </div>
)}
</div>
);
}

```

```
// Usage example
<MatchExplanation
  matchScore={94}
  reasons={[
    {
      factor: 'Sector Fit',
      score: 35,
      description: 'Both focused on Fintech - payments and lending',
      icon: <Target className="w-4 h-4" />,
    },
    {
      factor: 'Stage Alignment',
      score: 25,
      description: 'You\'re raising Seed, they invest at Seed stage',
      icon: <TrendingUp className="w-4 h-4" />,
    },
    {
      factor: 'Location Match',
      score: 15,
      description: 'Both based in Bangalore',
      icon: <MapPin className="w-4 h-4" />,
    },
    {
      factor: 'Thesis Alignment',
      score: 19,
      description: 'Their portfolio includes 3 fintech companies',
      icon: <Briefcase className="w-4 h-4" />,
    },
  ]}
/>
```

Tasks:

- Break down match score into factors
- Create explanation algorithm
- Build match explanation UI
- Add confidence indicators
- Show mutual interest indicators

2.5 Referral System (Days 43-45)

PRIORITY: P1 - HIGH

```
// backend/src/services/referral.ts

import { User, Referral } from '../models';

export class ReferralService {
```

```

async generateReferralCode(userId: string): Promise<string> {
  const user = await User.findById(userId);

  if (user.referralCode) {
    return user.referralCode;
  }

  // Generate unique code
  const code = this.generateUniqueCode(user.name);

  user.referralCode = code;
  await user.save();

  return code;
}

async trackReferral(referralCode: string, newUserId: string) {
  const referrer = await User.findOne({ referralCode });

  if (!referrer) {
    throw new Error('Invalid referral code');
  }

  // Create referral record
  await Referral.create({
    referrerId: referrer._id,
    referredId: newUserId,
    code: referralCode,
    status: 'pending',
    rewardStatus: 'pending',
  });

  // Update referrer stats
  await User.findByIdAndUpdate(referrer._id, {
    $inc: { 'stats.pendingReferrals': 1 },
  });
}

async completeReferral(newUserId: string) {
  const referral = await Referral.findOne({ referredId: newUserId });

  if (!referral || referral.status === 'completed') {
    return;
  }

  referral.status = 'completed';
  referral.completedAt = new Date();
  await referral.save();

  // Update referrer stats
  await User.findByIdAndUpdate(referral.referrerId, {

```

```

    $inc: {
      'stats.pendingReferrals': -1,
      'stats.completedReferrals': 1,
      'stats.totalReferralRewards': 1,
    },
  });

  // Grant rewards
  await this.grantRewards(referral);
}

private async grantRewards(referral: any) {
  // Double-sided reward

  // Referrer gets: 1 month free or extra matches
  await this.grantReferrerReward(referral.referrerId);

  // Referred user gets: Skip waitlist + bonus matches
  await this.grantReferredReward(referral.referredId);

  referral.rewardStatus = 'granted';
  await referral.save();
}

private async grantReferrerReward(userId: string) {
  // Add 1 month to subscription or add 5 bonus matches
  await User.findByIdAndUpdate(userId, {
    $inc: { 'subscription.bonusMatches': 5 },
  });

  await notificationService.sendNotification(userId, {
    type: 'referral_reward',
    title: 'Referral Reward Unlocked!',
    body: 'You earned 5 bonus matches for referring a new member.',
  });
}

private async grantReferredReward(userId: string) {
  // Skip waitlist if pending
  await User.findByIdAndUpdate(userId, {
    $set: { status: 'active' },
    $inc: { 'subscription.bonusMatches': 3 },
  });
}

private generateUniqueCode(name: string): string {
  const prefix = name.split(' ')[0].toUpperCase().slice(0, 4);
  const random = Math.random().toString(36).substring(2, 6).toUpperCase();
  return `${prefix}${random}`;
}

```



```

async getReferralStats(userId: string) {
  const user = await User.findById(userId);
  const referrals = await Referral.find({ referrerId: userId });

  return {
    code: user.referralCode,
    totalReferrals: referrals.length,
    completedReferrals: referrals.filter((r) => r.status === 'completed').length,
    pendingReferrals: referrals.filter((r) => r.status === 'pending').length,
    totalRewards: user.stats?.totalReferralRewards || 0,
    inviteUrl: `https://cleya.ai/signup?ref=${user.referralCode}`,
  };
}
}

```

```

// frontend/components/referral/ReferralDashboard.tsx

export function ReferralDashboard() {
  const [stats, setStats] = useState(null);
  const [copied, setCopied] = useState(false);

  useEffect(() => {
    fetch('/api/referrals/stats')
      .then((res) => res.json())
      .then((data) => setStats(data));
  }, []);

  const copyToClipboard = (text: string) => {
    navigator.clipboard.writeText(text);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };

  if (!stats) return <Skeleton className="h-64" />;

  return (
    <div className="bg-white rounded-lg shadow-md p-6">
      <h3 className="text-lg font-semibold mb-4">Invite Friends, Earn Rewards</h3>

      <div className="bg-gradient-to-r from-teal-500 to-teal-600 rounded-lg p-4 text-white mb-6">
        <p className="text-sm opacity-90 mb-2">Your Referral Code</p>
        <div className="flex items-center gap-3">
          <code className="text-2xl font-bold tracking-wider">{stats.code}</code>
          <Button
            size="sm"
            variant="secondary"
            onClick={() => copyToClipboard(stats.inviteUrl)}
          >
            {copied ? <Check className="w-4 h-4" /> : <Copy className="w-4 h-4" />}
          </Button>
        </div>
      </div>
    </div>
  );
}

```

```

</div>

<div className="grid grid-cols-3 gap-4 mb-6">
  <div className="text-center">
    <div className="text-2xl font-bold text-teal-600">{stats.totalReferrals}</div>
    <div className="text-xs text-gray-500">Total Invites</div>
  </div>
  <div className="text-center">
    <div className="text-2xl font-bold text-green-600">{stats.completedReferrals}</div>
    <div className="text-xs text-gray-500">Joined</div>
  </div>
  <div className="text-center">
    <div className="text-2xl font-bold text-purple-600">{stats.totalRewards}</div>
    <div className="text-xs text-gray-500">Rewards Earned</div>
  </div>
</div>

<div className="space-y-3">
  <p className="text-sm text-gray-600">Share your link:</p>
  <div className="flex gap-2">
    <Button
      variant="outline"
      className="flex-1"
      onClick={() => shareOnLinkedIn(stats.inviteUrl)}
    >
      <Linkedin className="w-4 h-4 mr-2" />
      <Linkedin />
    </Button>
    <Button
      variant="outline"
      className="flex-1"
      onClick={() => shareOnTwitter(stats.inviteUrl)}
    >
      <Twitter className="w-4 h-4 mr-2" />
      <Twitter />
    </Button>
    <Button
      variant="outline"
      className="flex-1"
      onClick={() => shareViaWhatsApp(stats.inviteUrl)}
    >
      <MessageCircle className="w-4 h-4 mr-2" />
      <WhatsApp />
    </Button>
  </div>
</div>

<div className="mt-6 p-4 bg-gray-50 rounded-lg">
  <h4 className="font-medium mb-2">How it works:</h4>
  <ul className="text-sm text-gray-600 space-y-1">
    <li>Share your unique link with friends</li>
  </ul>
</div>

```

```

    <li> They skip the waitlist when they sign up</li>
    <li> You earn 5 bonus matches per referral</li>
    <li> They get 3 bonus matches too!</li>
  </ul>
</div>
</div>
);
}

```

Tasks:

- Generate unique referral codes
- Track referral conversions
- Create referral dashboard
- Implement double-sided rewards
- Add social sharing buttons
- Show referral progress
- Send referral notifications

🟡 PHASE 3: GROWTH & SCALE (Days 46-90)

3.1 City-Specific Landing Pages (Days 46-52)

PRIORITY: P2 - MEDIUM

```

// frontend/app/cities/[city]/page.tsx

interface CityPageProps {
  params: { city: string };
}

const cityData = {
  bangalore: {
    name: 'Bangalore',
    tagline: 'The Silicon Valley of India',
    stats: {
      startups: '12,000+',
      funding: '$15B+',
      unicorns: '35+',
    },
    investors: ['Sequoia India', 'Accel', 'Blume Ventures'],
    sectors: ['SaaS', 'Fintech', 'E-commerce'],
    testimonials: [
      {
        name: 'Arjun M.',
        role: 'Founder',
        company: 'Fintech Startup',
        quote: 'Met my lead investor within a week of joining.',
      }
    ]
  }
}

```

```

    },
  ],
},
mumbai: {
  name: 'Mumbai',
  tagline: 'India\'s Financial Capital',
  stats: {
    startups: '8,000+',
    funding: '$8B+',
    unicorns: '20+',
  },
  investors: ['Matrix Partners', 'Lightspeed', 'Tiger Global'],
  sectors: ['Fintech', 'Media', 'Consumer'],
  testimonials: [...],
},
// Add Delhi, Hyderabad, Pune, Chennai
};

export default function CityPage({ params }: CityPageProps) {
  const city = cityData[params.city];

  if (!city) return notFound();

  return (
    <div className="min-h-screen">
      { /* Hero */ }
      <section className="bg-gradient-to-br from-teal-600 to-teal-800 text-white py-20">
        <div className="max-w-6xl mx-auto px-4">
          <div className="text-center">
            <h1 className="text-5xl font-bold mb-4">
              Connect with {city.name} Startups
            </h1>
            <p className="text-xl opacity-90 mb-2">{city.tagline}</p>
            <p className="text-lg opacity-75">
              Join {city.stats.startups} founders, {city.stats.funding} in funding
            </p>
            <Button size="lg" className="mt-8 bg-white text-teal-700 hover:bg-gray-100">
              Join {city.name} Community
            </Button>
          </div>
        </div>
      </section>

      { /* Stats */ }
      <section className="py-16 bg-gray-50">
        <div className="max-w-6xl mx-auto px-4">
          <div className="grid grid-cols-3 gap-8">
            <div className="text-center">
              <div className="text-4xl font-bold text-teal-600">{city.stats.startups}</div>
              <div className="text-gray-600">Startups</div>
            </div>

```

```

    <div className="text-center">
      <div className="text-4xl font-bold text-teal-600">{city.stats.funding}</div>
      <div className="text-gray-600">Funding Raised</div>
    </div>
    <div className="text-center">
      <div className="text-4xl font-bold text-teal-600">{city.stats.unicorns}</div>
      <div className="text-gray-600">Unicorns</div>
    </div>
  </div>
</div>
</div>
</section>

{/* Top Investors */}
<section className="py-16">
  <div className="max-w-6xl mx-auto px-4">
    <h2 className="text-3xl font-bold text-center mb-12">
      Top Investors in {city.name}
    </h2>
    <div className="flex flex-wrap justify-center gap-8">
      {city.investors.map((investor) => (
        <div key={investor} className="bg-white shadow-md rounded-lg px-6 py-4">
          <span className="font-semibold">{investor}</span>
        </div>
      ))}
    </div>
  </div>
</section>

{/* Sectors */}
<section className="py-16 bg-gray-50">
  <div className="max-w-6xl mx-auto px-4">
    <h2 className="text-3xl font-bold text-center mb-12">
      Hot Sectors in {city.name}
    </h2>
    <div className="flex flex-wrap justify-center gap-4">
      {city.sectors.map((sector) => (
        <Badge key={sector} className="text-lg px-4 py-2">
          {sector}
        </Badge>
      ))}
    </div>
  </div>
</section>

{/* Testimonials */}
<section className="py-16">
  <div className="max-w-6xl mx-auto px-4">
    <h2 className="text-3xl font-bold text-center mb-12">
      Success Stories from {city.name}
    </h2>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-8">

```

```

    {city.testimonials.map((t, i) => (
      <TestimonialCard key={i} {...t} />
    ))}
  </div>
</div>
</section>

{/* CTA */}
<section className="py-20 bg-teal-600 text-white">
  <div className="max-w-4xl mx-auto px-4 text-center">
    <h2 className="text-3xl font-bold mb-4">
      Ready to join {city.name}'s startup ecosystem?
    </h2>
    <p className="text-xl opacity-90 mb-8">
      Get introduced to the right investors, founders, and operators in {city.name}.
    </p>
    <Button size="lg" className="bg-white text-teal-700 hover:bg-gray-100">
      Apply for Early Access
    </Button>
  </div>
</section>
</div>
);
}
```

Tasks:

- Create city page template
- Add Bangalore, Mumbai, Delhi, Hyderabad, Pune, Chennai
- Add city-specific SEO metadata
- Create city-specific OG images
- Add local investor lists
- Add city-specific testimonials

3.2 Hindi Language Support (Days 53-58)

PRIORITY: P2 - MEDIUM

```
// frontend/i18n/config.ts

import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

const resources = {
  en: {
    translation: {
      hero: {
        title: 'Meet the right people. Faster.',

```

```

        subtitle: 'AI-powered warm intros for Indian startups',
        cta: 'Get Started',
      },
      nav: {
        howItWorks: 'How It Works',
        pricing: 'Pricing',
        about: 'About',
        login: 'Log In',
        signup: 'Get Started',
      },
      // ... more keys
    },
  },
  hi: {
    translation: {
      hero: {
        title: '000 00000 00 000000 00000 000',
        subtitle: '000000 000000000000 00 000 AI-00000000 00000 000000',
        cta: '0000 0000',
      },
      nav: {
        howItWorks: '00 0000 000 0000 00',
        pricing: '000000 000000000',
        about: '000000 0000 000',
        login: '000 00',
        signup: '0000 0000',
      },
      // ... more keys
    },
  },
};

i18n
  .use(initReactI18next)
  .init({
    resources,
    lng: 'en',
    fallbackLng: 'en',
    interpolation: {
      escapeValue: false,
    },
  });

export default i18n;

```

```

// frontend/components/LanguageSwitcher.tsx

export function LanguageSwitcher() {
  const { i18n } = useTranslation();

  const languages = [

```

```

    { code: 'en', label: 'English', flag: '$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/
    ↪ usr/share/docminer/emoji/1f1ee-1f1f3.png}}}}$$' },
    { code: 'hi', label: 'हिन्दी', flag: '$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/
    ↪ usr/share/docminer/emoji/1f1ee-1f1f3.png}}}}$$' },
  ];

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="ghost" size="sm">
          <Globe className="w-4 h-4 mr-2" />
          {languages.find((l) => l.code === i18n.language)?.label}
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent>
        {languages.map((lang) => (
          <DropdownMenuItem
            key={lang.code}
            onClick={() => i18n.changeLanguage(lang.code)}
            >
            <span className="mr-2">{lang.flag}</span>
            {lang.label}
          </DropdownMenuItem>
        ))}
      </DropdownMenuContent>
    </DropdownMenu>
  );
}

```

Tasks:

- Set up i18n framework
- Translate all UI strings
- Add language switcher
- Detect browser language
- Add RTL support (for future Arabic)

3.3 ML Spam Classifier (Days 59-65)

PRIORITY: P1 - HIGH

```

# ml-service/app.py

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np
from transformers import BertTokenizer, BertForSequenceClassification
import torch

```



```

app = FastAPI()

# Load pre-trained models
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('./spam_classifier_model')

class MessageRequest(BaseModel):
    content: str
    senderId: str
    recipientId: str
    messageType: str = 'text'

class SpamPrediction(BaseModel):
    isSpam: bool
    confidence: float
    reason: str
    action: str

@app.post("/predict", response_model=SpamPrediction)
async def predict_spam(request: MessageRequest):
    """
    Predict if a message is spam/fraudulent
    """
    # Preprocess
    inputs = tokenizer(
        request.content,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=512
    )

    # Predict
    with torch.no_grad():
        outputs = model(**inputs)
        probabilities = torch.softmax(outputs.logits, dim=1)
        spam_probability = probabilities[0][1].item()

    # Determine action
    if spam_probability > 0.9:
        return SpamPrediction(
            isSpam=True,
            confidence=spam_probability,
            reason="High confidence spam detection",
            action="block"
        )
    elif spam_probability > 0.7:
        return SpamPrediction(
            isSpam=True,
            confidence=spam_probability,

```

```

        reason="Suspicious content detected",
        action="quarantine"
    )
else:
    return SpamPrediction(
        isSpam=False,
        confidence=1 - spam_probability,
        reason="Content appears legitimate",
        action="allow"
    )

@app.post("/report")
async def report_spam(messageId: str, reporterId: str, reason: str):
    """
    Report a message as spam for retraining
    """
    # Store report for manual review and model retraining
    store_report(messageId, reporterId, reason)

    return {"status": "reported", "messageId": messageId}

# Pattern-based rules (fallback)
SUSPICIOUS_PATTERNS = [
    r'\b(?:send|transfer|pay)\s+(?:money|cash|funds)\b',
    r'\b(?:click|visit)\s+(?:here|link)\b',
    r'\b(?:urgent|immediate|asap)\s+(?:action|response)\b',
    r'\b(?:guaranteed|100%)\s+(?:returns|profits)\b',
    r'\$\d+(?:,\d{3})*(?:\.\d{2})?', # Dollar amounts
]

def rule_based_check(content: str) -> dict:
    """
    Fallback rule-based spam detection
    """
    import re

    score = 0
    matched_patterns = []

    for pattern in SUSPICIOUS_PATTERNS:
        if re.search(pattern, content, re.IGNORECASE):
            score += 0.2
            matched_patterns.append(pattern)

    return {
        "score": min(score, 1.0),
        "matchedPatterns": matched_patterns,
        "isSuspicious": score > 0.5
    }
}

```

// backend/src/middleware/spamFilter.ts

```

import { Request, Response, NextFunction } from 'express';

export async function spamFilter(req: Request, res: Response, next: NextFunction) {
  const { content, senderId, recipientId } = req.body;

  try {
    // Call ML service
    const response = await fetch(process.env.ML_SERVICE_URL + '/predict', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ content, senderId, recipientId }),
    });

    const prediction = await response.json();

    if (prediction.action === 'block') {
      // Log and block
      await logSpamAttempt(senderId, content, prediction);
      return res.status(403).json({
        error: 'Message blocked due to suspicious content',
        code: 'SPAM_DETECTED',
      });
    }

    if (prediction.action === 'quarantine') {
      // Queue for manual review
      await quarantineMessage(senderId, recipientId, content, prediction);
      return res.status(202).json({
        message: 'Message queued for review',
        code: 'PENDING_REVIEW',
      });
    }

    // Allow message
    req.body.spamScore = prediction.confidence;
    next();
  } catch (error) {
    console.error('Spam filter error:', error);
    // Fail open - allow message if ML service is down
    next();
  }
}

```

Tasks:

- Train spam classification model
- Set up ML service
- Create spam filter middleware
- Implement quarantine system

- Add manual review dashboard
 - Create user reporting system
 - Log all spam attempts
-

3.4 Accelerator Partnerships (Days 66-72)

PRIORITY: P2 - MEDIUM

```
// backend/src/services/partnerships.ts

export class PartnershipService {

  async onboardAccelerator(data: {
    name: string;
    logo: string;
    website: string;
    cohorts: Array<{
      name: string;
      startDate: Date;
      endDate: Date;
      companies: string[]; // User IDs
    }>;
    mentors: string[]; // User IDs
    investors: string[]; // User IDs
  }) {
    // Create accelerator profile
    const accelerator = await Accelerator.create({
      ...data,
      status: 'active',
      verified: true,
    });

    // Bulk verify all associated users
    await User.updateMany(
      { _id: { $in: [...data.companies, ...data.mentors, ...data.investors] } },
      {
        $set: {
          'verification.accelerator': {
            verified: true,
            acceleratorId: accelerator._id,
            verifiedAt: new Date(),
          }
        }
      }
    );

    return accelerator;
  }
}
```

```

async createCohortExclusiveMatches(acceleratorId: string, cohortId: string) {
  // Get all companies in cohort
  const cohort = await Cohort.findById(cohortId).populate('companies');

  // Get all mentors and investors
  const accelerator = await Accelerator.findById(acceleratorId)
    .populate('mentors investors');

  // Generate AI matches for each company
  for (const company of cohort.companies) {
    const matches = await matchingService.findMatches(company._id, {
      includeUsers: [...accelerator.mentors, ...accelerator.investors],
      limit: 10,
    });

    // Create intro requests
    for (const match of matches) {
      await introService.createIntroRequest({
        senderId: company._id,
        recipientId: match.userId,
        context: `Part of ${accelerator.name} ${cohort.name} cohort`,
        priority: 'high',
      });
    }
  }
}

```

Tasks:

- Partner with 3 accelerators (Antler, We Founder Circle, YC India)
- Create accelerator dashboard
- Bulk verify cohort companies
- Generate exclusive matches
- Track partnership ROI

3.5 Enterprise Tier (Days 73-80)

PRIORITY: P2 - MEDIUM

```

// backend/src/services/enterprise.ts

export class EnterpriseService {

  async createEnterpriseAccount(data: {
    companyName: string;
    adminEmail: string;
    teamSize: number;
    plan: 'growth' | 'scale' | 'custom';
  }) {

```

```

    }) {
      // Create enterprise account
      const enterprise = await Enterprise.create({
        ...data,
        status: 'pending',
        subscription: {
          plan: data.plan,
          startDate: new Date(),
          billingCycle: 'annual',
        },
      });

      // Send onboarding email
      await emailService.sendEnterpriseOnboarding(enterprise);

      // Schedule demo call
      await calendarService.scheduleDemoCall(enterprise.adminEmail);

      return enterprise;
    }

    async provisionEnterpriseFeatures(enterpriseId: string) {
      const enterprise = await Enterprise.findById(enterpriseId);

      // Enable features based on plan
      const features = {
        growth: ['api_access', 'advanced_analytics', 'dedicated_support'],
        scale: ['api_access', 'advanced_analytics', 'dedicated_support', 'custom_matching', 'sso'],
        custom: ['all_features', 'sla', 'custom_contract'],
      };

      enterprise.enabledFeatures = features[enterprise.plan];
      await enterprise.save();

      // Create API keys
      if (enterprise.enabledFeatures.includes('api_access')) {
        const apiKey = await this.generateAPIKey(enterpriseId);
        enterprise.apiKey = apiKey;
        await enterprise.save();
      }

      return enterprise;
    }

    async generateAPIKey(enterpriseId: string): Promise<string> {
      const crypto = require('crypto');
      const key = crypto.randomBytes(32).toString('hex');

      await APIKey.create({
        enterpriseId,
        key: crypto.createHash('sha256').update(key).digest('hex'),
      });
    }
  }
}

```

```

    permissions: ['read:profiles', 'read:matches', 'write:intros'],
    ratelimit: 10000, // requests per hour
    createdAt: new Date(),
  });

  return `cleya_live_${key}`;
}
}

```

```

// backend/src/routes/api.ts

import { Router } from 'express';
import { authenticateAPIKey } from '../middleware/auth';

const router = Router();

// All API routes require authentication
router.use(authenticateAPIKey);

// Get profiles
router.get('/profiles', async (req, res) => {
  const { role, sector, stage, limit = 20 } = req.query;

  const profiles = await User.find({
    role,
    'company.sector': sector,
    'company.stage': stage,
    status: 'active',
  })
    .select('name profile headline company verification')
    .limit(Number(limit));

  res.json({ profiles });
});

// Get matches for a profile
router.get('/matches/:userId', async (req, res) => {
  const { userId } = req.params;
  const { limit = 10 } = req.query;

  const matches = await matchingService.findMatches(userId, {
    limit: Number(limit),
  });

  res.json({ matches });
});

// Request intro
router.post('/intros', async (req, res) => {
  const { senderId, recipientId, context } = req.body;

```

```

const intro = await introService.createIntroRequest({
  senderId,
  recipientId,
  context,
  source: 'api',
});

res.json({ intro });
});

export default router;

```

Tasks:

- Create enterprise onboarding flow
- Build API infrastructure
- Add SSO integration
- Create usage analytics
- Add dedicated support channel
- Implement custom matching rules

3.6 Content Marketing (Days 81-90)

PRIORITY: P2 - MEDIUM

```

// frontend/app/blog/page.tsx

export default function BlogPage() {
  const posts = [
    {
      slug: 'how-to-raise-seed-funding-india-2024',
      title: 'How to Raise Seed Funding in India: Complete Guide 2024',
      excerpt: 'A step-by-step guide for founders looking to raise their first round in the Indian
        ↪ ecosystem.',
      author: 'Rahul Sharma',
      date: '2024-03-15',
      category: 'Fundraising',
      readTime: '12 min read',
      image: '/blog/seed-funding.jpg',
    },
    {
      slug: 'top-angel-investors-india-fintech',
      title: 'Top 50 Angel Investors in Indian Fintech',
      excerpt: 'Our curated list of active angel investors funding fintech startups in India.',
      author: 'Priya Patel',
      date: '2024-03-10',
      category: 'Investors',
      readTime: '8 min read',
    },
  ];
}

```



```

        image: '/blog/angel-investors.jpg',
    },
    // More posts...
];

return (
  <div className="max-w-6xl mx-auto py-16 px-4">
    <div className="text-center mb-12">
      <h1 className="text-4xl font-bold mb-4">Startup Insights</h1>
      <p className="text-xl text-gray-600">
        Guides, research, and insights from India's startup ecosystem
      </p>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
      {posts.map((post) => (
        <BlogCard key={post.slug} {...post} />
      ))}
    </div>
  </div>
);
}

```

```

// frontend/app/blog/[slug]/page.tsx

export async function generateMetadata({ params }: { params: { slug: string } }) {
  const post = await getBlogPost(params.slug);

  return {
    title: `${post.title} | Cleya.ai Blog`,
    description: post.excerpt,
    openGraph: {
      images: [{ url: post.image }],
    },
  };
}

export default async function BlogPostPage({ params }: { params: { slug: string } }) {
  const post = await getBlogPost(params.slug);

  return (
    <article className="max-w-3xl mx-auto py-16 px-4">
      <header className="mb-8">
        <div className="flex gap-2 mb-4">
          <Badge>{post.category}</Badge>
          <span className="text-gray-500">{post.readTime}</span>
        </div>
        <h1 className="text-4xl font-bold mb-4">{post.title}</h1>
        <div className="flex items-center gap-4 text-gray-600">
          <Avatar>
            <AvatarImage src={post.authorImage} />

```

```

        <AvatarFallback>{post.author[0]}</AvatarFallback>
      </Avatar>
      <div>
        <div className="font-medium">{post.author}</div>
        <div>{new Date(post.date).toLocaleDateString('en-IN')}</div>
      </div>
    </div>
  </header>

  <div className="prose prose-lg max-w-none">
    <MDXRemote source={post.content} />
  </div>

  { /* CTA */ }
  <div className="mt-12 p-6 bg-teal-50 rounded-lg">
    <h3 className="text-xl font-semibold mb-2">
      Ready to put these insights into action?
    </h3>
    <p className="text-gray-600 mb-4">
      Join Cleya.ai and connect with the right investors for your startup.
    </p>
    <Button>Get Started Free</Button>
  </div>
</article>
);
}
```

Tasks:

- Create blog with 10 SEO-optimized posts
- Add founder success stories
- Create investor spotlights
- Add industry reports
- Implement email newsletter
- Add social sharing

 SUCCESS METRICS

Days 1-30 Targets

- 0 critical security vulnerabilities
- Google indexing started
- Pricing page live with 3 tiers
- 20% conversion improvement

Days 31-60 Targets

- In-app messaging launched

- 3x platform stickiness
- 50% weekly active users
- 1L MRR

Days 61-90 Targets

- 20,000 total members
 - 5L MRR
 - 0.4 viral coefficient
 - 50% organic traffic growth
-

🔧 TECH STACK

Frontend

- **Framework:** Next.js 14 (App Router)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **Components:** shadcn/ui
- **State:** Zustand
- **Forms:** React Hook Form + Zod
- **Realtime:** Socket.io Client

Backend

- **Runtime:** Node.js
- **Framework:** Express.js
- **Language:** TypeScript
- **Database:** MongoDB (Mongoose)
- **Cache:** Redis
- **Realtime:** Socket.io
- **Queue:** Bull (Redis)

Infrastructure

- **Hosting:** Vercel (frontend), Railway/Render (backend)
- **Database:** MongoDB Atlas
- **CDN:** Cloudflare
- **Storage:** AWS S3
- **Email:** SendGrid/Resend
- **Monitoring:** LogRocket, Sentry

ML/AI

- **Framework:** Python + FastAPI
 - **Models:** BERT for spam, custom for matching
 - **Deployment:** Hugging Face + AWS SageMaker
-

💰 BUDGET BREAKDOWN

Category	Cost ()
Development Team (3 devs × 3 months)	15,00,000
Infrastructure (Vercel, Railway, MongoDB)	50,000
ML Service (AWS SageMaker)	1,00,000
Third-party APIs (SendGrid, etc.)	30,000
Design & Assets	50,000
Marketing (initial)	2,00,000
Contingency	70,000
TOTAL	20,00,000

⚠️ RISK MITIGATION

Risk	Mitigation
Team burnout	Phased rollout, clear priorities
Technical debt	Code reviews, testing, documentation
Security breaches	Regular audits, penetration testing
Low adoption	Beta testing, feedback loops
Competition	Speed to market, differentiation

📞 SUPPORT & RESOURCES

Documentation

- API Docs: <https://docs.cleya.ai>
- Design System: <https://design.cleya.ai>
- Engineering Wiki: Notion/Github Wiki

Communication

- Daily standups: 10 AM IST
- Weekly demos: Friday 4 PM IST
- Slack: #cleya-engineering

Tools

- Project Management: Linear/Jira
- Design: Figma
- Code: GitHub
- CI/CD: GitHub Actions

✅ FINAL CHECKLIST

Pre-Launch (Day 90)

- All P0 bugs fixed
- Security audit passed
- Performance benchmarks met
- User testing completed
- Content marketing live
- Support docs ready
- Monitoring dashboards up
- Rollback plan documented

Post-Launch

- Daily metrics review
- Weekly user feedback sessions
- Monthly roadmap reviews
- Quarterly strategic planning

Let's build India's best startup networking platform! 🚀